Additional Research Objectives for AI-Chat Pre-Wiring and Pre-Assembly

Overview

The existing CHATDIRECTOR v1 AI-Chat Pre-Wiring Research Plan already defines core objectives around contract surfaces, invariants/metrics mapping, lifecycle phases, validation loops, Dead-Ledger readiness, and telemetry. To further improve pre-wiring and pre-assembly for AI-generated codes and files, additional research objectives can extend this foundation into discovery contracts, agent profiles, capability catalogs, cross-repo routing, and regression harnesses that shape how AI-Chat interacts with the constellation before any file is emitted.[1][2] [^3]

1. Discovery Contracts as a Hard Precondition

Research should formalize a small, strict `discovery-contract.v1` schema that records exactly which spine documents, repo manifests, registries, and capability catalogs an AI session has loaded before authoring. CHATDIRECTOR would refuse any `chatDirectorRequestv1` that does not reference a fresh, valid discovery contract, ensuring AI-Chat cannot operate on stale routing or invariant data.[^3]

This objective includes defining Rust and Lua helpers to validate discovery contracts, wiring them into `plan` so that request normalization always checks discovery first, and adding CI tests that prove no authoring flow can bypass this gate.[^3]

2. Capability Catalog Surfaces for AI-Chat

Pre-wiring can be strengthened by exposing a read-only capability catalog derived from the schema spine and manifests, describing available object kinds, invariants, metrics, phases, tiers, and routing rules in a prompt-safe JSON view. Research should specify how `ChatDirector.catalog` (Rust) and `H.Discovery.capabilityCatalog` (Lua) serialize this data so AI-Chat can query "what is legal here?" before it proposes any artifact.[2][3]

Key tasks include defining the catalog schema, clarifying which invariant/metric descriptions are safe for prompts, and designing telemetry to measure how catalog usage affects validation success rates and misrouting incidents.[^2]

3. AI-Safe Authoring Profiles and Per-Agent Budgets

Beyond a single global contract surface, pre-wiring should be shaped by per-agent `ai-safe-authoring-profile-v1` cards that encode numeric caps and floors on invariants, metrics, tiers, phases, and repo access for each AI role. Research needs to determine how CHATDIRECTOR reads these profiles, intersects them with spine bands, and uses them during `plan` to produce agent-specific `generationGuide` ranges instead of generic defaults.[^4]

This includes studying how flags like `allowsClamp` and `allowsSchemaUpgrade` interact with pre-wiring and promotion rules, and how different profile settings for Architect vs Implementer agents influence CI health and Dead-Ledger eligibility over time.[^4]

4. Enhanced Cross-Repo Routing and Tier Gating for AI-Chat

Existing manifest rules already route objectKind+tier to repos and enforce policies like `OneFilePerRequest`, but pre-wiring can be extended with explicit `aiAuthoringRules` and `authoringHints` blocks inside repo manifests. Research should define how these fields constrain AI-Chat pre-wiring (e.g., forbidding raw narrative in Tier 1, requiring `deadledgerRef` placeholders in Dead-Ledger repos, or redirecting low-RWF artifacts to quarantine paths).[^2]

Objectives include specifying a `RouteExplanation` shape that `plan` can surface to AI-Chat, capturing chosen repo, tier, allowed schemas, example paths, and policy notes so the model can align its generation with manifest intent before validation.[^2]

5. History-Layer Coupling and SHCI in Pre-Wiring

The current plan treats SHCI and history coupling mostly as invariant bands, but pre-wiring can more tightly bind AI-generated entities to concrete history bundles and Dead-Ledger references. Research should define how `chatDirectorRequestv1` embeds required history sources (e.g., Black-Archivum bundles, region IDs) and SHCI targets, and how `plan` consults history-layer indices before suggesting skeletons.[3][4] [^2]

This objective also covers rules for when high-SHCI or high-CIC artifacts must be routed to lab-tier repos, and how Dead-Ledger `deadledgerRef` placeholders are pre-assembled for later proof attachment without ever embedding proofs themselves.[^4]

6. Regression Harness and AI Failure Corpus for Pre-Wiring

Pre-wiring quality depends on learning from failures. Research should define an AI failure regression corpus and golden test harness focused specifically on pre-wiring behaviors: misrouted artifacts, invariant/metric band violations, phase/tier mismatches, and discovery-contract omissions.[^2]

Tasks include designing a small family of `.json` test cases derived from real AI-Chat errors, capturing original `chatDirectorRequestv1`, diagnostics from the validation pipeline, and corrected payloads, then wiring these into CI so future changes to `plan`, manifests, or spines cannot regress established behaviors.[^2]

7. IDE and Editor Integration Contracts

To support pre-assembly in tools beyond chat, research should define minimal contracts and protocols for IDE/editor plugins (Unreal, Godot, VS Code) that call CHATDIRECTOR's `plan` and `validate-response` on behalf of users. This objective covers how editor context (open file, selection, repo root) is normalized into `chatDirectorRequestv1`, how pre-wiring hints are shown in the UI, and how one-file-per-request and phase gates are enforced in authoring workflows.[3][2]

The goal is to ensure that pre-wiring logic is identical across chat integrations and IDEs, using the same discovery contracts, capability catalogs, manifests, and agent profiles, so AI-Chat and human-in-editor flows share a single governance surface.[^2]

8. Telemetry-Driven Tuning of Pre-Wiring Parameters

The existing plan already calls for `GenerationMetrics` logging, but additional research can focus on telemetry strategies specific to pre-wiring: catalog usage, discovery-contract freshness, profile violations, routing corrections, and retry counts per error code. Objectives include defining schema fields for these signals, aggregating them by objectKind+tier+repo, and analyzing how they correlate with CI pass rates and runtime metrics like UEC and ARR.[1][4]

These findings can then drive automated or semi-automated updates to spine safe defaults, manifest `authoringHints`, profile bounds, and `generationGuide` behaviors so that pre-wiring improves continuously based on observed AI-Chat performance rather than static design assumptions.[^1]

9. Prism Envelope Extensions for Pre-Wiring Intent

Finally, research should consider extending prism envelopes so they carry explicit pre-wiring intent: planned invariants/metrics bands, discovery-contract IDs, agent profile IDs, and Dead-Ledger surface declarations, not just raw payloads and `schemaref`. This would let validation and Dead-Ledger tooling verify that emitted files match the pre-wired plan and that no out-of-contract invariants, metrics, or surfaces slip through.[4][3]

The objective is to design these extensions so they remain envelope-only (no proofs or trauma data), keep `additionalProperties: false`, and align with existing CHATDIRECTOR request/response schemas and Dead-Ledger routing policies.[^4]

References

1. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md - You are right that director-comfort-policy.requiredCooldownRatio uses lowDetMinutes and highDetMinut...
2. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
3. this-research-focuses-on-devel-Eh3ThsQBQg.r6HDAZZhx5w.md
4. formal-definition-and-integrat-V8UK1uYHRKez9vQ0Xfga2A.md